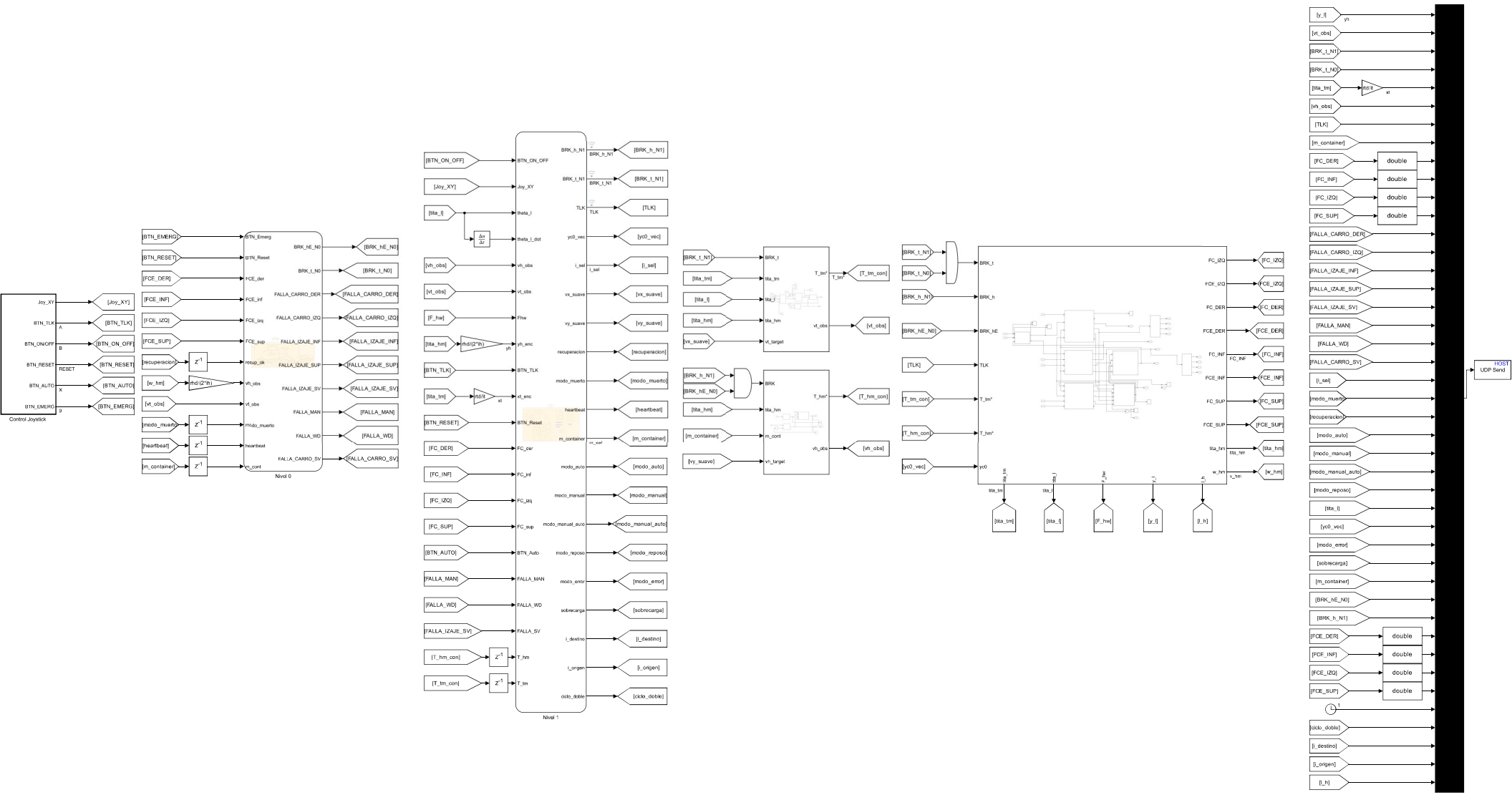
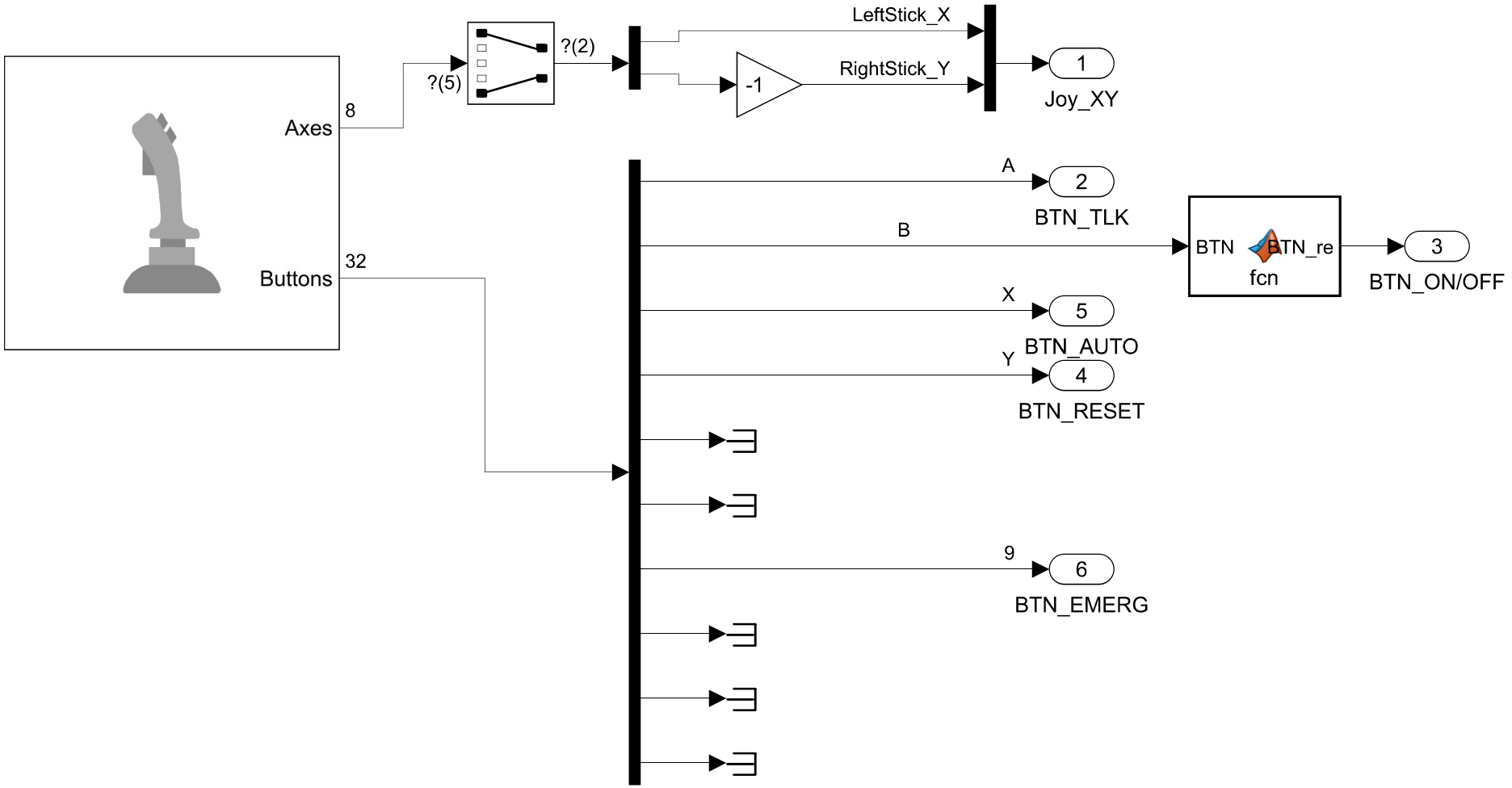
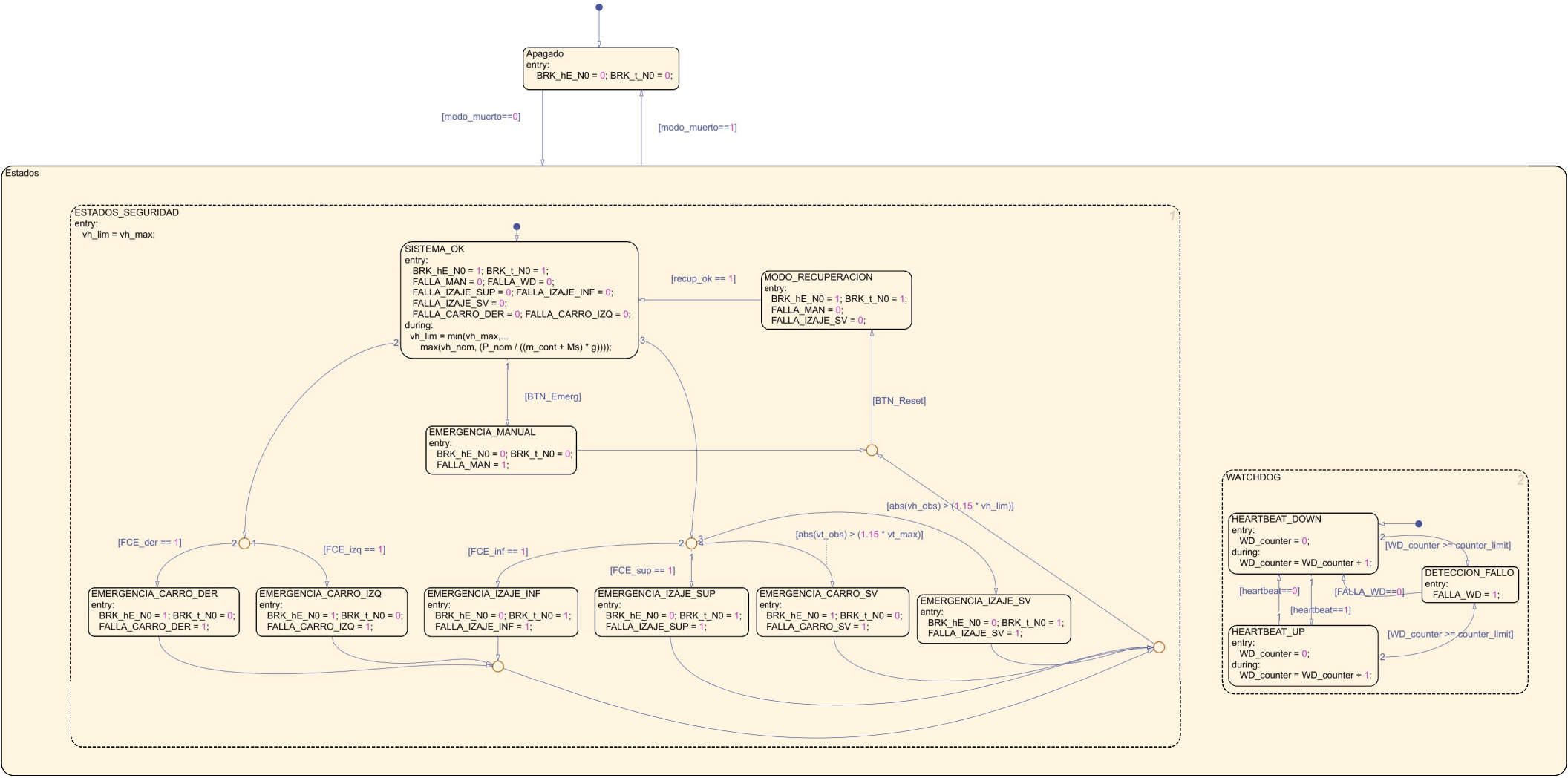
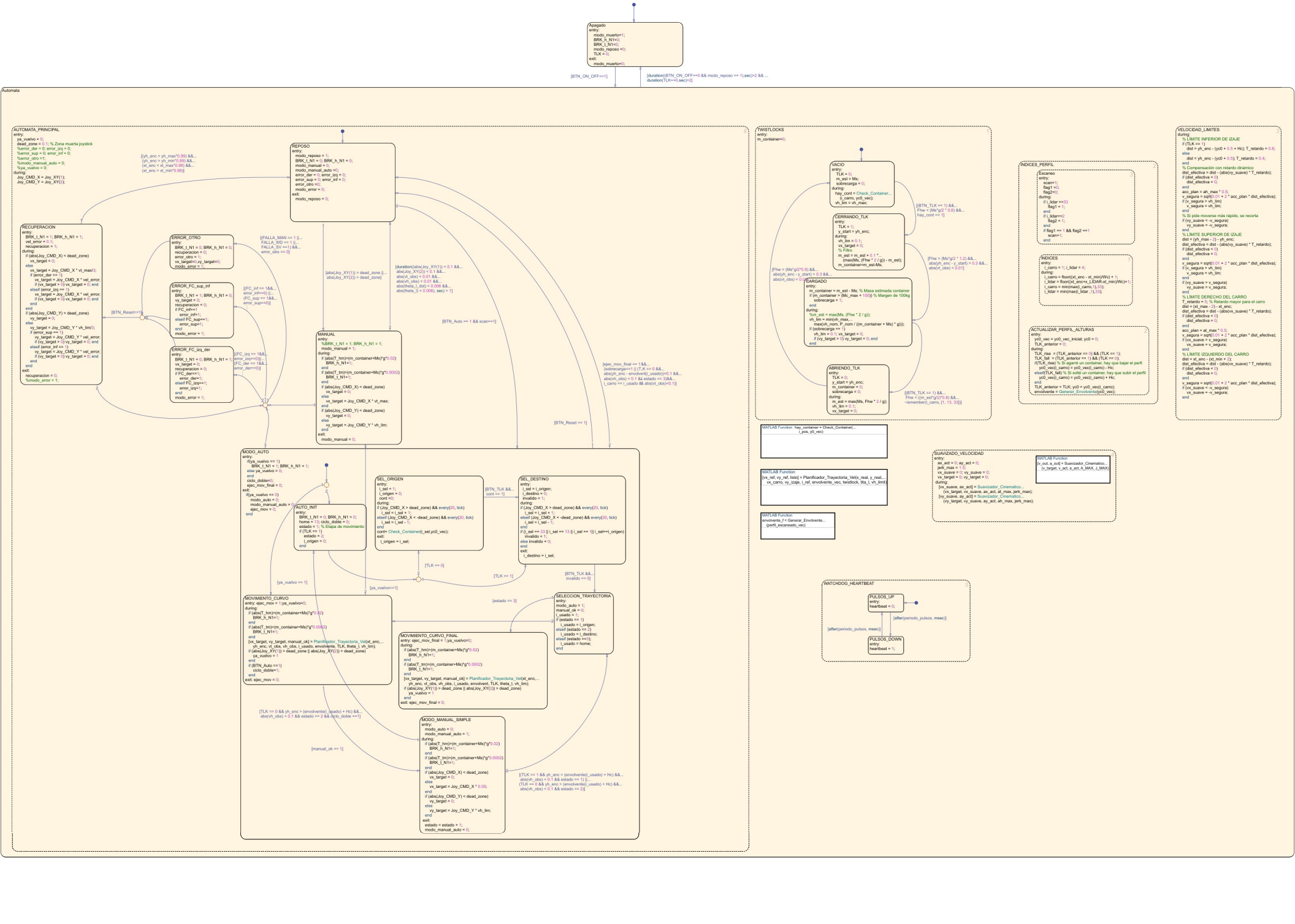
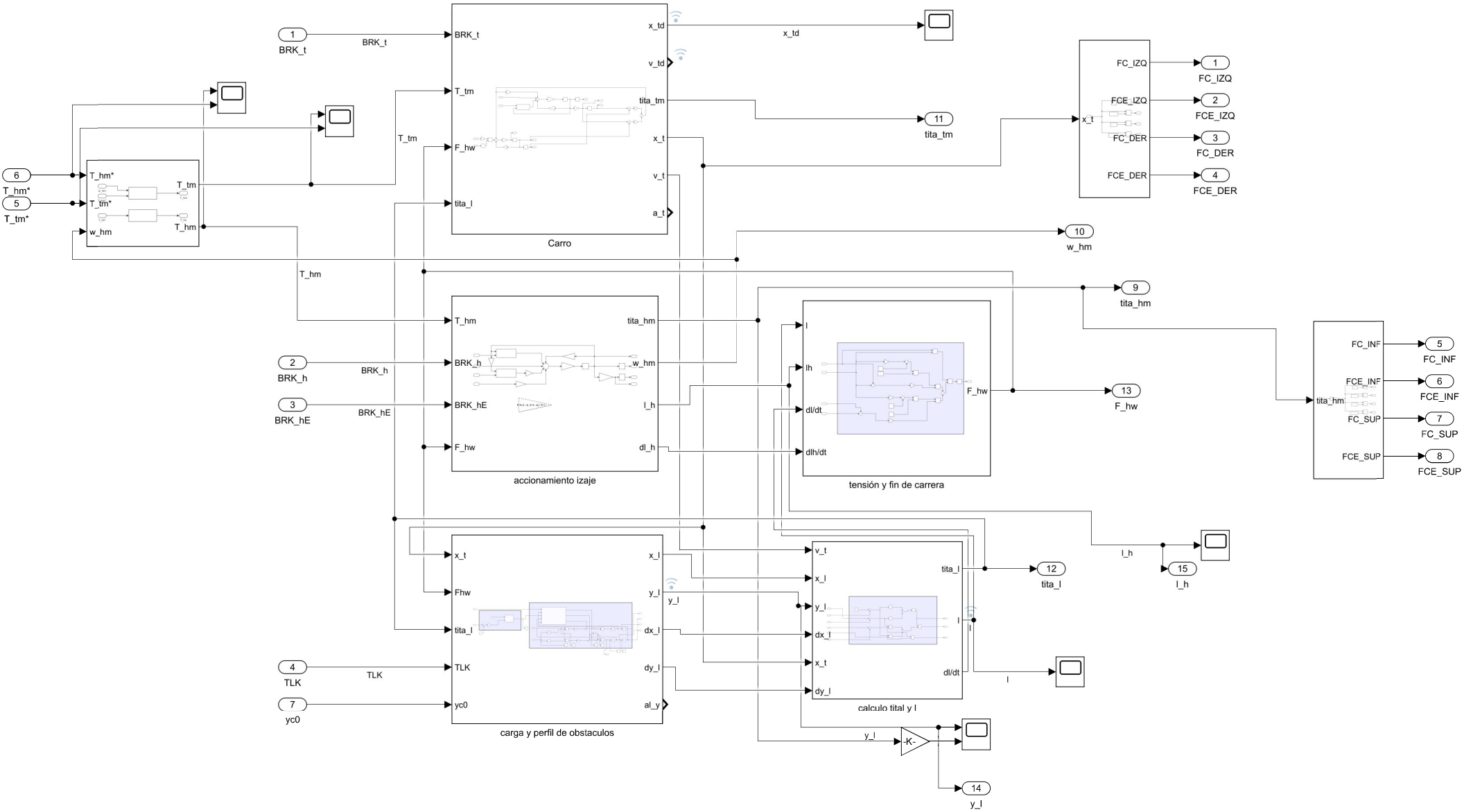

Control Joystick
Joy_XY
BTN_TLK
BTN_ON/OFF
BTN_RESET
BTN_AUTO
BTN_EMERG
[Joy_XY]
[BTN_TLK]
[BTN_ON_OFF]
[BTN_RESET]
[BTN_AUTO]
[BTN_EMERG]
A
B
RESET
X
Y
[BTN_EMERG]
[BTN_RESET]
[FCE_DER]
[FCE_INF]
[FCE_IZQ]
[FCE_SUP]
[recuperacion]
[w_hm]
[vt_obs]
[modo_muerto]
[heartbeat]
[m_container]
BTN_Emerg
BTN_Reset
FCE_der
FCE_inf
FCE_izq
FCE_sup
recup_ok
vh_obs
vt_obs
modo_muerto
heartbeat
m_cont
BRK_hE_N0
BRK_t_N0
FALLA_CARRO_DER
FALLA_CARRO_IZQ
FALLA_IZAJE_INF
FALLA_IZAJE_SUP
FALLA_IZAJE_SV
FALLA_MAN
FALLA_WD
FALLA_CARRO_SV
[BRK_hE_N0]
[BRK_t_N0]
[FALLA_CARRO_DER]
[FALLA_CARRO_IZQ]
[FALLA_IZAJE_INF]
[FALLA_IZAJE_SUP]
[FALLA_IZAJE_SV]
[FALLA_MAN]
[FALLA_WD]
[FALLA_CARRO_SV]
Nivel 0
[BTN_ON_OFF]
[Joy_XY]
[tita_l]
[vh_obs]
[vt_obs]
[F_hw]
[tita_hm]
[BTN_TLK]
[tita_tm]
[BTN_RESET]
[FC_DER]
[FC_INF]
[FC_IZQ]
[FC_SUP]
[BTN_AUTO]
[FALLA_MAN]
[FALLA_WD]
[FALLA_IZAJE_SV]
[T_hm_con]
[T_tm_con]
BTN_ON_OFF
Joy_XY
theta_l
theta_l_dot
vh_obs
vt_obs
Fhw
yh_enc
BTN_TLK
vt_enc
BTN_Reset
FC_der
FC_inf
FC_izq
FC_sup
BTN_Auto
FALLA_MAN
FALLA_WD
FALLA_SV
T_hm
T_tm
BRK_h_N1
BRK_t_N1
TLK
yc0_vec
i_sel
vx_suave
vy_suave
recuperacion
modo_muerto
heartbeat
m_container
modo_auto
modo_manual
modo_manual_auto
modo_reposo
modo_error
sobrecarga
i_destino
i_origen
ciclo_doble
[BRK_h_N1]
[BRK_t_N1]
[TLK]
[yc0_vec]
[i_sel]
[vx_suave]
[vy_suave]
[recuperacion]
[modo_muerto]
[heartbeat]
[m_container]
[modo_auto]
[modo_manual]
[modo_manual_auto]
[modo_reposo]
[modo_error]
[sobrecarga]
[i_destino]
[i_origen]
[ciclo_doble]
Nivel 1
[BRK_t_N1]
[tita_tm]
[tita_l]
[tita_hm]
[vx_suave]
[BRK_h_N1]
[BRK_hE_N0]
[tita_hm]
[m_container]
[vy_suave]
BRK_t
tita_tm
tita_l
tita_hm
vt_target
T_tm*
vt_obs
BRK
tita_hm
m_cont
vh_target
T_hm*
vh_obs
[T_tm_con]
[vt_obs]
[T_hm_con]
[vh_obs]
[BRK_t_N1]
[BRK_t_N0]
[BRK_h_N1]
[BRK_hE_N0]
[TLK]
[T_tm_con]
[T_hm_con]
[yc0_vec]
BRK_t
BRK_h
BRK_hE
TLK
T_tm*
T_hm*
yc0
tita_tm
tita_l
F_hw
y_l
l_h
FC_IZQ
FCE_IZQ
FC_DER
FCE_DER
FC_INF
FCE_INF
FC_SUP
FCE_SUP
tita_hm
w_hm
[FC_IZQ]
[FCE_IZQ]
[FC_DER]
[FCE_DER]
[FC_INF]
[FCE_INF]
[FC_SUP]
[FCE_SUP]
[tita_hm]
[w_hm]
[tita_tm]
[tita_l]
[F_hw]
[y_l]
[l_h]
[y_l]
[vt_obs]
[BRK_t_N1]
[BRK_t_N0]
[tita_tm]
[vh_obs]
[TLK]
[m_container]
[FC_DER]
[FC_INF]
[FC_IZQ]
[FC_SUP]
[FALLA_CARRO_DER]
[FALLA_CARRO_IZQ]
[FALLA_IZAJE_INF]
[FALLA_IZAJE_SUP]
[FALLA_IZAJE_SV]
[FALLA_MAN]
[FALLA_WD]
[FALLA_CARRO_SV]
[i_sel]
[modo_muerto]
[recuperacion]
[modo_auto]
[modo_manual]
[modo_manual_auto]
[modo_reposo]
[tita_l]
[yc0_vec]
[modo_error]
[sobrecarga]
[m_container]
[BRK_hE_N0]
[BRK_h_N1]
[FCE_DER]
[FCE_INF]
[FCE_IZQ]
[FCE_SUP]
[ciclo_doble]
[i_destino]
[i_origen]
[l_h]
yh
xt
double
HOST
UDP Send

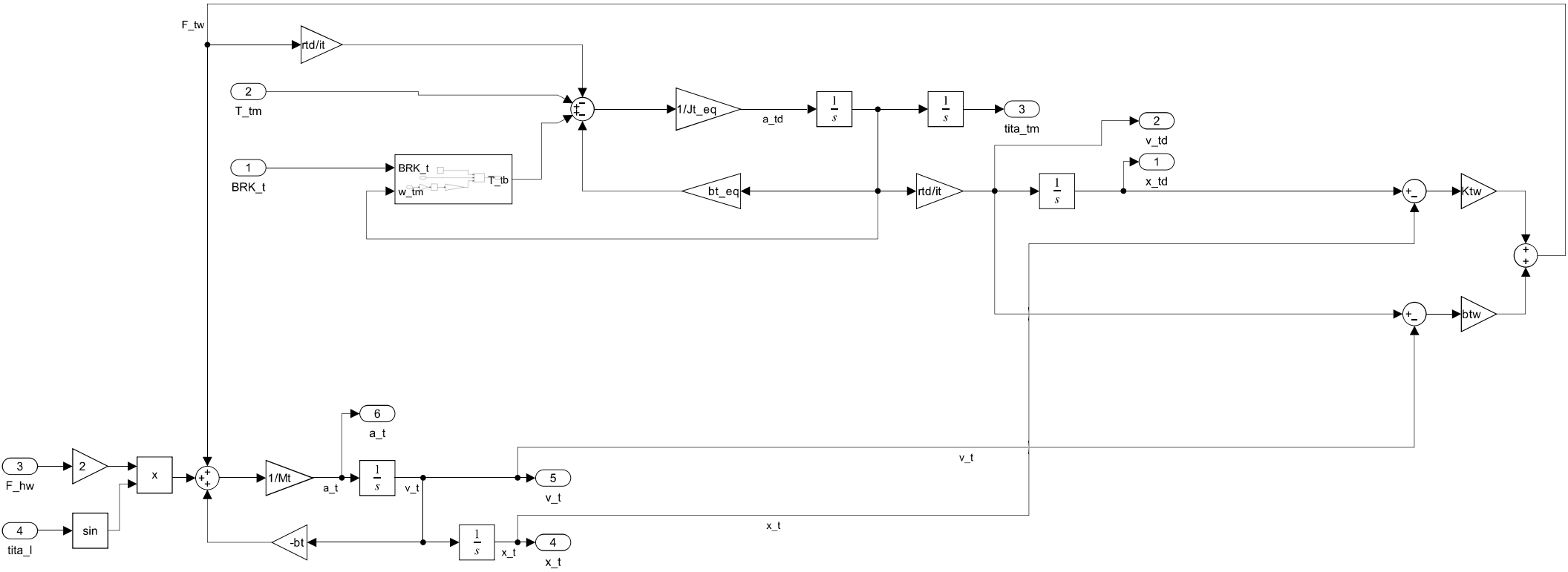

Axes
8
Buttons
32
?(5)
?(2)
LeftStick_X
RightStick_Y
-1
1
Joy_XY
A
2
BTN_TLK
B
BTN
BTN_re
fcn
3
BTN_ON/OFF
X
5
BTN_AUTO
Y
4
BTN_RESET
9
6
BTN_EMERG

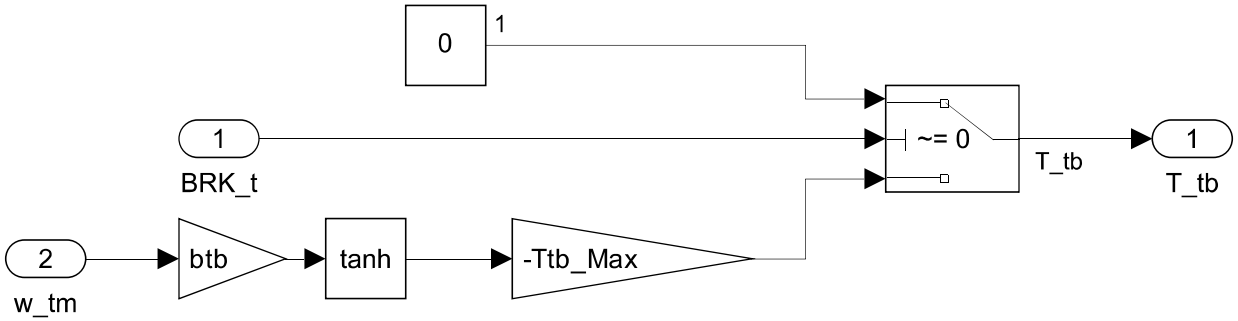
```
function BTN_re = fcn(BTN)
    persistent prev;
    persistent state;
    if isempty(prev)
        prev = 0;
        state = 0;
    end
    if BTN == 1 && prev == 0
        state = double(~logical(state));
    end
    BTN_re = state;
    prev = double(BTN);
end
```

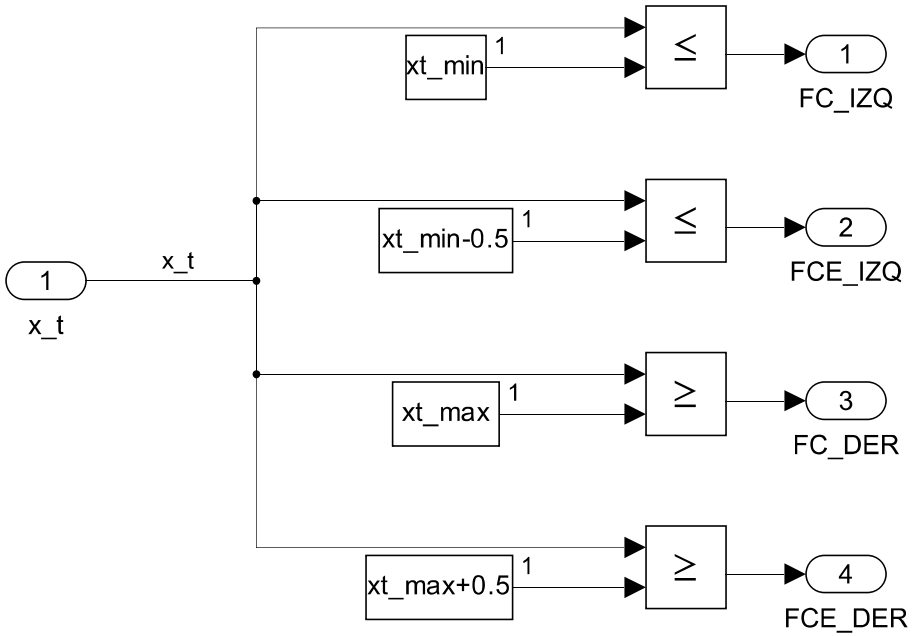

Apagado
entry:
BRK_hE_N0 = 0; BRK_t_N0 = 0;
[modo_muerto==0]
[modo_muerto==1]
Estados
ESTADOS_SEGURIDAD
entry:
vh_lim = vh_max;
SISTEMA_OK
entry:
BRK_hE_N0 = 1; BRK_t_N0 = 1;
FALLA_MAN = 0; FALLA_WD = 0;
FALLA_IZAJE_SUP = 0; FALLA_IZAJE_INF = 0;
FALLA_IZAJE_SV = 0;
FALLA_CARRO_DER = 0; FALLA_CARRO_IZQ = 0;
during:
vh_lim = min(vh_max,...
max(vh_nom, (P_nom / ((m_cont + Ms) * g))));
[recup_ok == 1]
MODO_RECUPERACION
entry:
BRK_hE_N0 = 1; BRK_t_N0 = 1;
FALLA_MAN = 0;
FALLA_IZAJE_SV = 0;
[BTN_Emerg]
[BTN_Reset]
EMERGENCIA_MANUAL
entry:
BRK_hE_N0 = 0; BRK_t_N0 = 0;
FALLA_MAN = 1;
[abs(vh_obs) > (1.15 * vh_lim)]
[abs(vt_obs) > (1.15 * vt_max)]
[FCE_der == 1]
[FCE_izq == 1]
[FCE_inf == 1]
[FCE_sup == 1]
EMERGENCIA_CARRO_DER
entry:
BRK_hE_N0 = 1; BRK_t_N0 = 0;
FALLA_CARRO_DER = 1;
EMERGENCIA_CARRO_IZQ
entry:
BRK_hE_N0 = 1; BRK_t_N0 = 0;
FALLA_CARRO_IZQ = 1;
EMERGENCIA_IZAJE_INF
entry:
BRK_hE_N0 = 0; BRK_t_N0 = 1;
FALLA_IZAJE_INF = 1;
EMERGENCIA_IZAJE_SUP
entry:
BRK_hE_N0 = 0; BRK_t_N0 = 1;
FALLA_IZAJE_SUP = 1;
EMERGENCIA_CARRO_SV
entry:
BRK_hE_N0 = 1; BRK_t_N0 = 0;
FALLA_CARRO_SV = 1;
EMERGENCIA_IZAJE_SV
entry:
BRK_hE_N0 = 0; BRK_t_N0 = 1;
FALLA_IZAJE_SV = 1;
WATCHDOG
HEARTBEAT_DOWN
entry:
WD_counter = 0;
during:
WD_counter = WD_counter + 1;
[WD_counter >= counter_limit]
DETECCION_FALLO
entry:
FALLA_WD = 1;
[heartbeat==0]
[FALLA_WD==0]
[heartbeat==1]
HEARTBEAT_UP
entry:
WD_counter = 0;
during:
WD_counter = WD_counter + 1;
[WD_counter >= counter_limit]

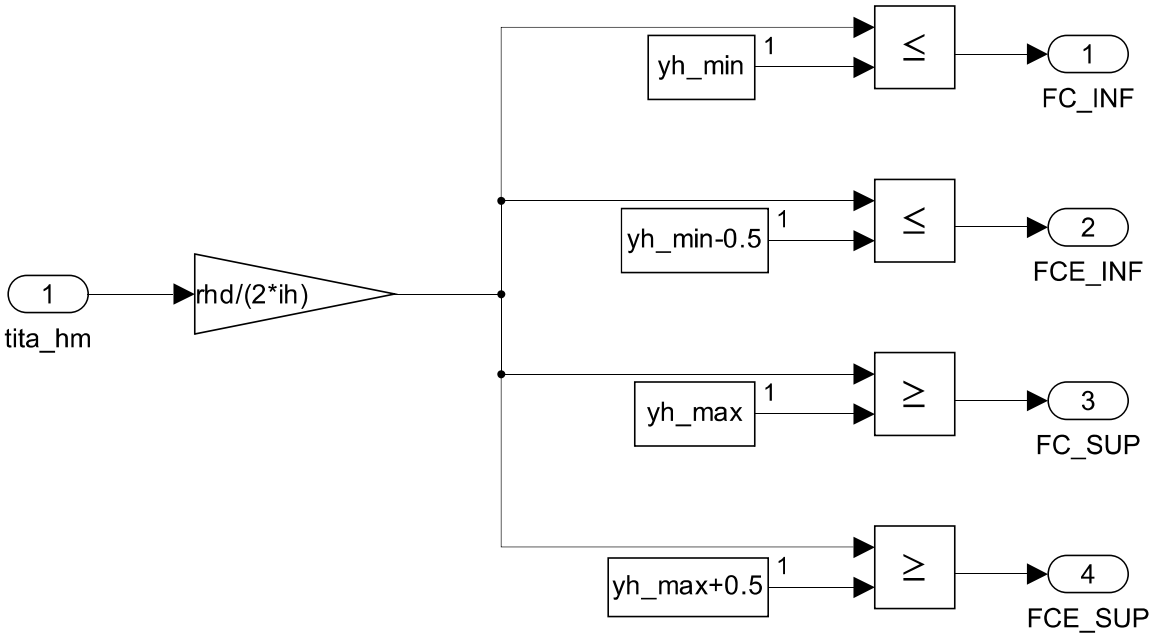

Apagado
entry:
modo_muerto=1;
BRK_h_N1=0;
BRK_t_N1=0;
modo_reposo =0;
TLK = 0;
exit:
modo_muerto=0;
[BTN_ON_OFF==1]
[duration(([BTN_ON_OFF==0 && modo_reposo == 1),sec)>2 && ...
duration(TLK==0,sec)>2]
Automata
AUTOMATA_PRINCIPAL
entry:
ya_vuelvo = 0;
dead_zone = 0.1; % Zona muerta joystick
%error_der = 0; error_izq = 0;
%error_sup = 0; error_inf = 0;
%error_otro =1;
%modo_manual_auto = 0;
%ya_vuelvo = 0;
during:
Joy_CMD_X = Joy_XY(1);
Joy_CMD_Y = Joy_XY(2);
REPOSO
entry:
modo_reposo = 1;
BRK_t_N1 = 0; BRK_h_N1 = 0;
modo_manual = 0;
modo_manual_auto =0;
error_der = 0; error_izq = 0;
error_sup = 0; error_inf = 0;
error_otro =0;
modo_error = 0;
exit:
modo_reposo = 0;
[(yh_enc < yh_max*0.99) &&...
(yh_enc > yh_min*0.99) &&...
(xt_enc < xt_max*0.99) &&...
(xt_enc > xt_min*0.99)]
RECUPERACION
entry:
BRK_t_N1 = 1; BRK_h_N1 = 1;
vel_error = 0.1;
recuperacion = 1;
during:
if (abs(Joy_CMD_X) < dead_zone)
vx_target = 0;
else
vx_target = Joy_CMD_X * vt_max/3;
if (error_der == 1)
vx_target = Joy_CMD_X * vel_error;
if (vx_target > 0) vx_target = 0; end
elseif (error_izq == 1)
vx_target = Joy_CMD_X * vel_error;
if (vx_target < 0) vx_target = 0; end
end
end
if (abs(Joy_CMD_Y) < dead_zone)
vy_target = 0;
else
vy_target = Joy_CMD_Y * vh_lim/3;
if (error_sup == 1)
vy_target = Joy_CMD_Y * vel_error;
if (vy_target > 0) vy_target = 0; end
elseif (error_inf == 1)
vy_target = Joy_CMD_Y * vel_error;
if (vy_target < 0) vy_target = 0; end
end
end
exit:
recuperacion = 0;
%modo_error = 1;
[BTN_Reset==1]
ERROR_OTRO
entry:
BRK_t_N1 = 0; BRK_h_N1 = 0;
recuperacion = 0;
error_otro = 1;
vx_target=0; vy_target=0;
modo_error = 1;
[||FALLA_MAN == 1 ||...
FALLA_WD == 1 ||...
FALLA_SV ==1) &&...
error_otro == 0]
ERROR_FC_sup_inf
entry:
BRK_t_N1 = 1; BRK_h_N1 = 0;
vy_target = 0;
recuperacion = 0;
if FC_inf==1
error_inf=1;
elseif FC_sup==1;
error_sup=1;
end
modo_error = 1;
[(FC_inf == 1&&...
error_inf==0) ||...
(FC_sup == 1&&...
error_sup==0)]
ERROR_FC_izq_der
entry:
BRK_t_N1 = 0; BRK_h_N1 = 1;
vx_target = 0;
recuperacion = 0;
if FC_der==1;
error_der=1;
elseif FC_izq==1;
error_izq=1;
end
modo_error = 1;
[(FC_izq == 1&&...
error_izq==0)||...
(FC_der == 1&&...
error_der==0)]
[abs(Joy_XY(1)) > dead_zone ||...
abs(Joy_XY(2)) > dead_zone]
[duration(abs(Joy_XY(1)) < 0.1 &&...
abs(Joy_XY(2)) < 0.1 &&...
abs(vt_obs) < 0.01 &&...
abs(vh_obs) < 0.01 &&...
abs(theta_t_dot) < 0.006 &&...
abs(theta_t) < 0.006), sec) > 1]
[BTN_Auto == 1 && scan==1]
MANUAL
entry:
%BRK_t_N1 = 1; BRK_h_N1 = 1;
modo_manual = 1;
during:
if (abs(T_hm)>(m_container+Ms)*g*0.02)
BRK_h_N1=1;
end
if (abs(T_tm)>(m_container+Ms)*g*0.0002)
BRK_t_N1=1;
end
if (abs(Joy_CMD_X) < dead_zone)
vx_target = 0;
else
vx_target = Joy_CMD_X * vt_max;
end
if (abs(Joy_CMD_Y) < dead_zone)
vy_target = 0;
else
vy_target = Joy_CMD_Y * vh_lim;
end
exit:
modo_manual = 0;
[BTN_Reset == 1]
[ejec_mov_final == 1 &&...
(sobrecarga==1 || (T_K == 0 &&...
abs(yh_enc - envolvente(i_usado))<0.1 &&...
abs(vh_obs) < 0.1 && estado == 3)&&...
i_carro == i_usado && abs(xt_obs)<0.1)]
MODO_AUTO
entry:
if((ya_vuelvo == 1)
BRK_t_N1 = 1; BRK_h_N1 = 1;
else ya_vuelvo = 0;
end
ciclo_doble=0;
ejec_mov_final = 0;
exit:
if((ya_vuelvo == 0)
modo_auto = 0;
modo_manual_auto = 0;
ejec_mov = 0;
end
AUTO_INIT
entry:
BRK_t_N1 = 0; BRK_h_N1 = 0;
home = 1; ciclo_doble = 0;
estado = 1; % Etapa de movimiento
if (TLK == 1)
estado = 2;
i_origen = 0;
end
SEL_ORIGEN
entry:
i_sel = 1;
i_origen = 0;
cont =0;
during:
if (Joy_CMD_X > dead_zone) && every(20, tick)
i_sel = i_sel + 1;
elseif (Joy_CMD_X < -dead_zone) && every(20, tick)
i_sel = i_sel - 1;
end
cont= Check_Container(i_sel,yc0_vec);
exit:
i_origen = i_sel;
[BTN_TLK &&...
cont == 1]
SEL_DESTINO
entry:
i_sel = i_origen;
i_destino = 0;
invalido = 1;
during:
if (Joy_CMD_X > dead_zone) && every(20, tick)
i_sel = i_sel + 1;
elseif (Joy_CMD_X < -dead_zone) && every(20, tick)
i_sel = i_sel - 1;
end
if (i_sel == 23 || i_sel == 13 || i_sel == 1|| i_sel==i_origen)
invalido = 1;
else invalido = 0;
end
exit:
i_destino = i_sel;
[TLK == 0]
[TLK == 1]
[BTN_TLK &&...
invalido == 0]
[ya_vuelvo == 1]
[ya_vuelvo==1]
MOVIMIENTO_CURVO
entry: ejec_mov = !ya_vuelvo=0;
during:
if (abs(T_hm)>(m_container+Ms)*g*0.02)
BRK_h_N1=1;
end
if (abs(T_tm)>(m_container+Ms)*g*0.0002)
BRK_t_N1=1;
end
[vx_target, vy_target, manual_ok] = Planificador_Trayectoria_Vel(xt_enc,...
yh_enc, vt_obs, vh_obs, i_usado, envolvente, TLK, theta_t, vh_lim);
if (abs(Joy_XY(1)) > dead_zone || abs(Joy_XY(2)) > dead_zone)
ya_vuelvo = !
end
if (BTN_Auto ==1)
ciclo_doble=1;
end
exit: ejec_mov = 0;
[estado == 3]
SELECCION_TRAYECTORIA
entry:
modo_auto = 1;
manual_ok = 0;
i_usado = 1;
if (estado == 1)
i_usado = i_origen;
elseif (estado == 2)
i_usado = i_destino;
elseif (estado ==3);
i_usado = home;
end
MOVIMIENTO_CURVO_FINAL
entry: ejec_mov_final = !ya_vuelvo=0;
during:
if (abs(T_hm)>(m_container+Ms)*g*0.02)
BRK_h_N1=1;
end
if (abs(T_tm)>(m_container+Ms)*g*0.0002)
BRK_t_N1=1;
end
[vx_target, vy_target, manual_ok] = Planificador_Trayectoria_Vel(xt_enc,...
yh_enc, vt_obs, vh_obs, i_usado, envolvent, TLK, theta_t, vh_lim);
if (abs(Joy_XY(1)) > dead_zone || abs(Joy_XY(2)) > dead_zone)
ya_vuelvo = !
end
exit: ejec_mov_final = 0;
[TLK == 0 && yh_enc > (envolvente(i_usado) + Hc) &&...
abs(vh_obs) < 0.1 && estado == 2 && ciclo_doble ==1]
[manual_ok == 1]
MODO_MANUAL_SIMPLE
entry:
modo_auto = 0;
modo_manual_auto = 1;
during:
if (abs(T_hm)>(m_container+Ms)*g*0.02)
BRK_h_N1=1;
end
if (abs(T_tm)>(m_container+Ms)*g*0.0002)
BRK_t_N1=1;
end
if (abs(Joy_CMD_X) < dead_zone)
vx_target = 0;
else
vx_target = Joy_CMD_X * 0.05;
end
if (abs(Joy_CMD_Y) < dead_zone)
vy_target = 0;
else
vy_target = Joy_CMD_Y * vh_lim;
end
exit:
estado = estado + 1;
modo_manual_auto = 0;
[(TLK == 1 && yh_enc > (envolvente(i_usado) + Hc) &&...
abs(vh_obs) < 0.1 && estado == 1) ||...
(TLK == 0 && yh_enc > (envolvente(i_usado) + Hc) &&...
abs(vh_obs) < 0.1 && estado == 2)]
TWISTLOCKS
entry:
m_container=0;
VACIO
entry:
TLK = 0;
m_est = Ms;
sobrecarga = 0;
during:
hay_cont = Check_Container...
(i_carro, yc0_vec);
vh_lim = vh_max;
[(BTN_TLK == 1) &&...
Fhw < (Ms*g/2 * 0.8) &&...
hay_cont == 1]
CERRANDO_TLK
entry:
TLK = 1;
y_start = yh_enc;
during:
vh_lim = 0.1;
vx_target = 0;
% Filtro
m_est = m_est + 0.1 *...
(max(Ms, (Fhw * 2 / g)) - m_est);
m_container=m_est-Ms;
[Fhw > (Ms*g/2 * 1.2) &&...
abs(yh_enc - y_start) > 0.3 &&...
abs(vt_obs) < 0.01]
[Fhw > (Ms*g/2*0.8) &&...
abs(yh_enc - y_start) < 0.3 &&...
abs(vt_obs) < 0.01]
EMPUJANDO
entry:
m_container = m_est - Ms; % Masa estimada container
if (m_container > (Mc_max + 100)) % Margen de 100kg
sobrecarga = 1;
end
during:
%m_est = max(Ms, (Fhw * 2 / g));
vh_lim = min(vh_max,
max(vh_nom, P_nom / ((m_container + Ms) * g)));
if (sobrecarga == 1)
vh_lim = 0.1; vx_target = 0;
if (vy_target > 0) vy_target = 0; end
end
ABRIENDO_TLK
entry:
TLK = 0;
y_start = yh_enc;
m_container = 0;
sobrecarga = 0;
during:
m_est = max(Ms, Fhw * 2 / g);
vh_lim = 0.1;
vx_target = 0;
[(BTN_TLK == 1) &&...
Fhw < ((m_est*g/2)*0.8) &&...
~ismember(i_carro, [1, 13, 33])]
MATLAB Function hay_container = Check_Container(...
i_pos, yc_vec)
MATLAB Function
[vx_ref, vy_ref, listo] = Planificador_Trayectoria_Vel(x_real, y_real,...
vx_carro, vy_izaje, i_ref, envolvente_vec, twistlock, tita_l, vh_lim)
MATLAB Function
envolvente_f = Generar_Envolvente...
(perfil_escaneado_vec)
INDICES_PERFIL
Escaneo
entry:
scan=1;
flag1 =0;
flag2=0;
during:
if (i_lidar==33
flag1 = 1;
end
if i_lidarr=2
flag2 = 1;
end
if flag1 == 1 && flag2 ==1
scan=1;
end
INDICES
entry:
i_carro = 1; i_lidar = 4;
during:
i_carro = floor((xt_enc - xt_min)/Wc) + 1;
i_lidar = floor((xt_enc+x_LIDAR-xt_min)/Wc)+1;
i_carro = min(max(i_carro,1),33);
i_lidar = min(max(i_lidar ,1),33);
ACTUALIZAR_PERFIL_ALTURAS
entry:
yc0_vec = yc0_vec_inicial; yc0 = 0;
TLK_anterior = 0;
during:
TLK_rise = (TLK_anterior == 0) && (TLK == 1);
TLK_fall = (TLK_anterior == 1) && (TLK == 0);
if(TLK_rise) % Si agarré un container, hay que bajar el perfil
yc0_vec(i_carro) = yc0_vec(i_carro) - Hc;
elseif(TLK_fall) % Si acabo un container, hay que subir el perfil
yc0_vec(i_carro) = yc0_vec(i_carro) + Hc;
end
TLK_anterior = TLK; yc0 = yc0_vec(i_carro);
envolvente = Generar_Envolvente(yc0_vec);
SUAVIZADO_VELOCIDAD
entry:
ax_act = 0; ay_act = 0;
jerk_max = 1.5;
vx_suave = 0; vy_suave = 0;
vx_target = 0; vy_target = 0;
during:
[vx_suave, ax_act] = Suavizador_Cinematico_...
(vx_target, vx_suave, ax_act, at_max, jerk_max);
[vy_suave, ay_act] = Suavizador_Cinematico_...
(vy_target, vy_suave, ay_act, ah_max, jerk_max);
MATLAB Function
[v_out, a_out] = Suavizador_Cinematico...
(v_target, v_act, a_act, A_MAX, J_MAX)
WATCHDOG_HEARTBEAT
PULSOS_UP
entry:
heartbeat = 0;
[after(periodo_pulsos, msec)]
[after(periodo_pulsos, msec)]
PULSOS_DOWN
entry:
heartbeat = 1;
VELOCIDAD_LIMITES
during:
% LÍMITE INFERIOR DE IZAJE
if (TLK == 1)
dist = yh_enc - (yc0 + 0.5 + Hc); T_retardo = 0.8;
else
dist = yh_enc - (yc0 + 0.5); T_retardo = 0.4;
end
% Compensación con retardo dinámico
dist_efectiva = dist - (abs(vy_suave) * T_retardo);
if (dist_efectiva < 0)
dist_efectiva = 0;
end
acc_plan = ah_max * 0.5;
v_segura = sqrt(0.01 + 2 * acc_plan * dist_efectiva);
if (v_segura > vh_lim)
v_segura = vh_lim;
end
% Si pide moverse más rápido, se recorta
if (vy_suave < -v_segura)
vy_suave = -v_segura;
end
% LÍMITE SUPERIOR DE IZAJE
dist = (yh_max - 2) - yh_enc;
dist_efectiva = dist - (abs(vy_suave) * T_retardo);
if (dist_efectiva < 0)
dist_efectiva = 0;
end
v_segura = sqrt(0.01 + 2 * acc_plan * dist_efectiva);
if (v_segura > vh_lim)
v_segura = vh_lim;
end
if (vy_suave > v_segura)
vy_suave = v_segura;
end
% LÍMITE DERECHO DEL CARRO
T_retardo = 5; % Retardo mayor para el carro
dist = (xt_max - 2) - xt_enc;
dist_efectiva = dist - (abs(vx_suave) * T_retardo);
if (dist_efectiva < 0)
dist_efectiva = 0;
end
acc_plan = at_max * 0.5;
v_segura = sqrt(0.01 + 2 * acc_plan * dist_efectiva);
if (vx_suave > v_segura)
vx_suave = v_segura;
end
% LÍMITE IZQUIERDO DEL CARRO
dist = xt_enc - (xt_min + 2);
dist_efectiva = dist - (abs(vx_suave) * T_retardo);
if (dist_efectiva < 0)
dist_efectiva = 0;
end
v_segura = sqrt(0.01 + 2 * acc_plan * dist_efectiva);
if (vx_suave < -v_segura)
vx_suave = -v_segura;
end

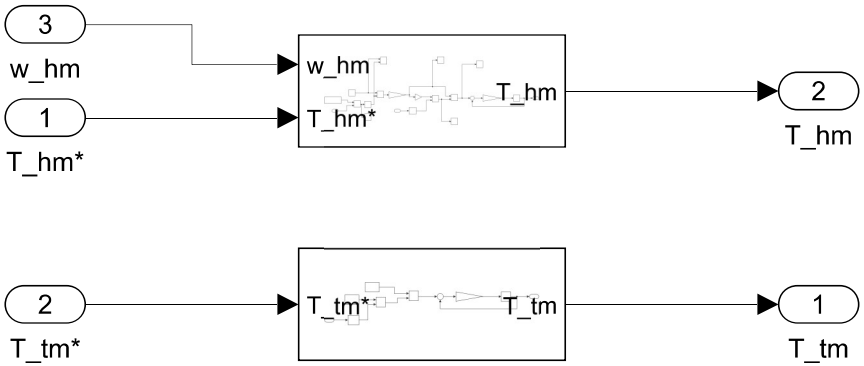

1
BRK_t
BRK_t
x_td
v_td
T_tm
tita_tm
F_hw
x_t
v_t
a_t
tita_l
Carro
x_td
11
tita_tm
FC_IZQ
FCE_IZQ
x_t
FC_DER
FCE_DER
1
FC_IZQ
2
FCE_IZQ
3
FC_DER
4
FCE_DER
6
T_hm*
5
T_tm*
T_hm*
T_tm*
w_hm
T_tm
T_hm
T_tm
T_hm
10
w_hm
9
tita_hm
2
BRK_h
3
BRK_hE
BRK_h
BRK_hE
T_hm
BRK_h
BRK_hE
F_hw
tita_hm
w_hm
I_h
dl_h
accionamiento izaje
I
Ih
dI/dt
dIh/dt
F_hw
tensión y fin de carrera
13
F_hw
tita_hm
FC_INF
FCE_INF
FC_SUP
FCE_SUP
5
FC_INF
6
FCE_INF
7
FC_SUP
8
FCE_SUP
x_t
Fhw
tita_l
TLK
yc0
x_l
y_l
dx_l
dy_l
al_y
carga y perfil de obstaculos
4
TLK
TLK
7
yc0
v_t
x_l
y_l
dx_l
x_t
dy_l
tita_l
I
dI/dt
calculo tital y I
12
tita_l
I_h
15
I_h
I
y_l
-K-
14
y_l

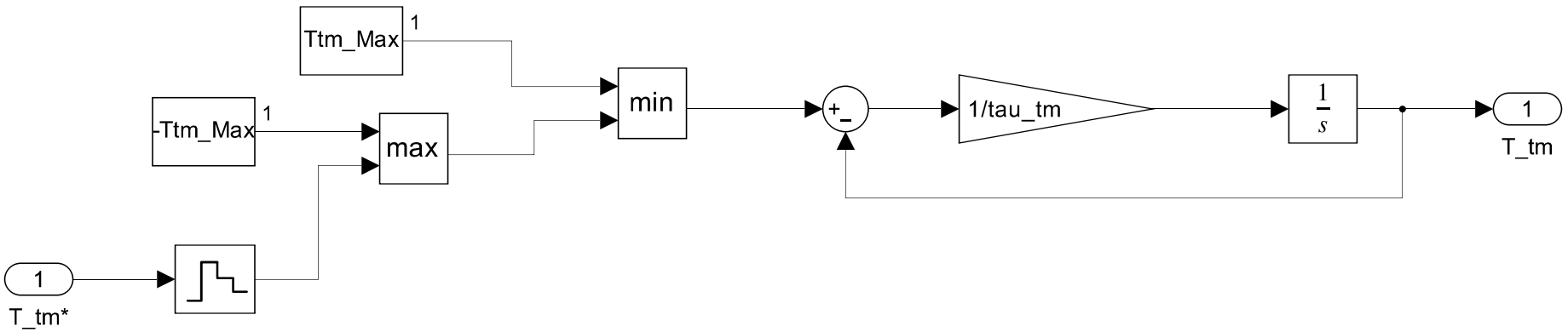

F_tw
rtd/it
2
T_tm
1
BRK_t
BRK_t
w_tm
T_tb
1/Jt_eq
a_td
3
tita_tm
2
v_td
1
x_td
bt_eq
rtd/it
Ktw
btw
6
a_t
3
F_hw
2
x
1/Mt
a_t
v_t
5
v_t
4
tita_l
sin
-bt
x_t
4
x_t
v_t
x_t

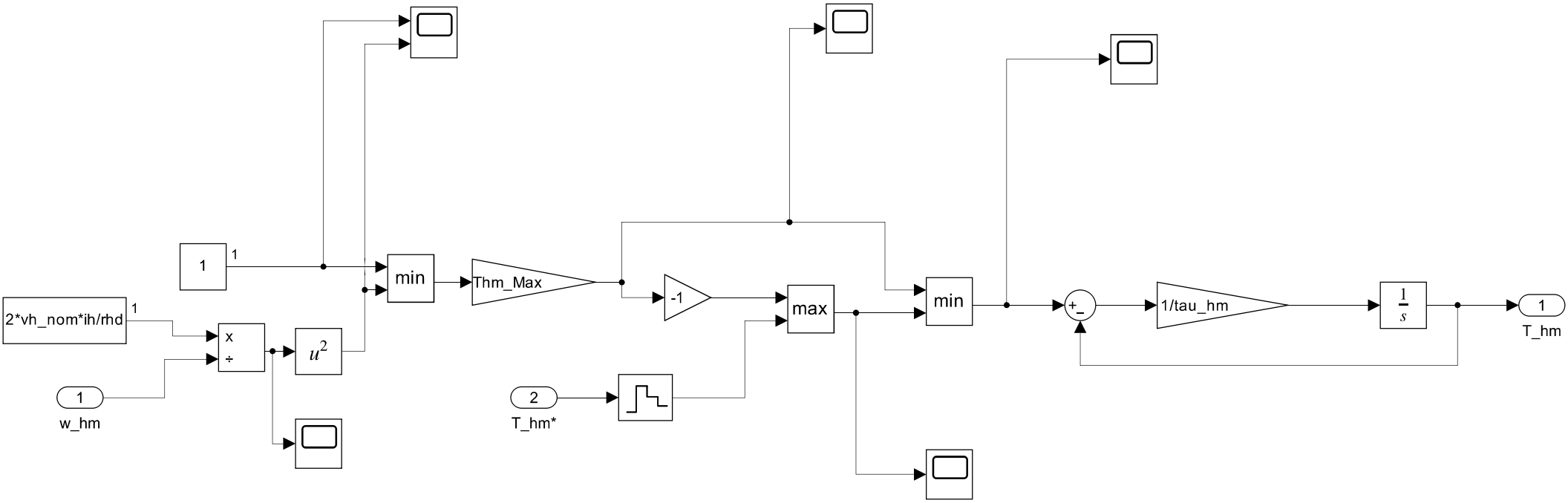

0
1
1
BRK_t
~= 0
T_tb
1
T_tb
2
w_tm
btb
tanh
-Ttb_Max

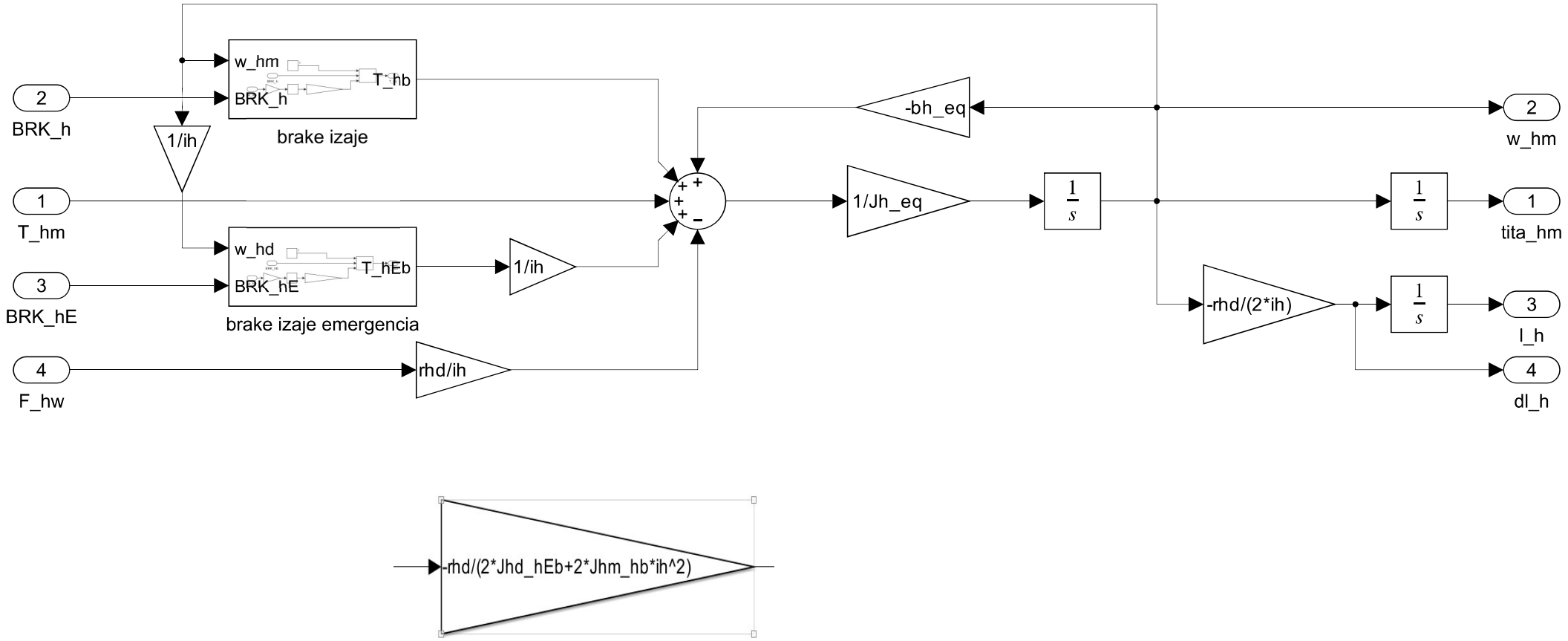

1
x_t
x_t
xt_min
1
≤
1
FC_IZQ
xt_min-0.5
1
≤
2
FCE_IZQ
xt_max
1
≥
3
FC_DER
xt_max+0.5
1
≥
4
FCE_DER


1
tita_hm
rhd/(2*ih)
yh_min
1
≤
1
FC_INF
yh_min-0.5
1
≤
2
FCE_INF
yh_max
1
≥
3
FC_SUP
yh_max+0.5
1
≥
4
FCE_SUP

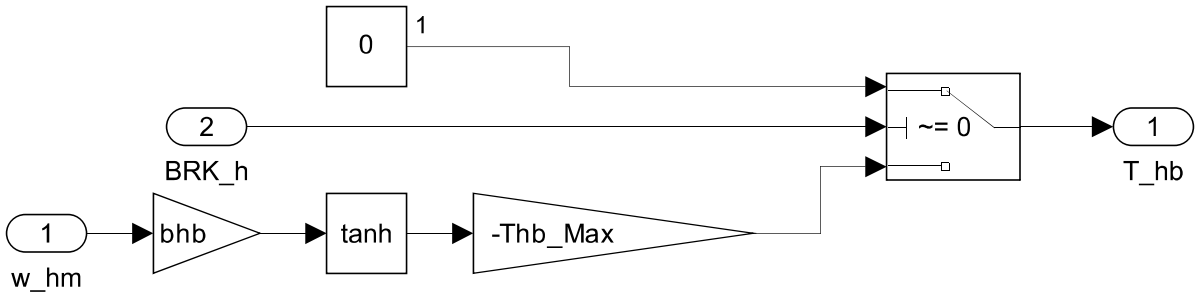

3
w_hm
1
T_hm*
w_hm
T_hm*
T_hm
2
T_hm
2
T_tm*
T_tm*
T_tm
1
T_tm

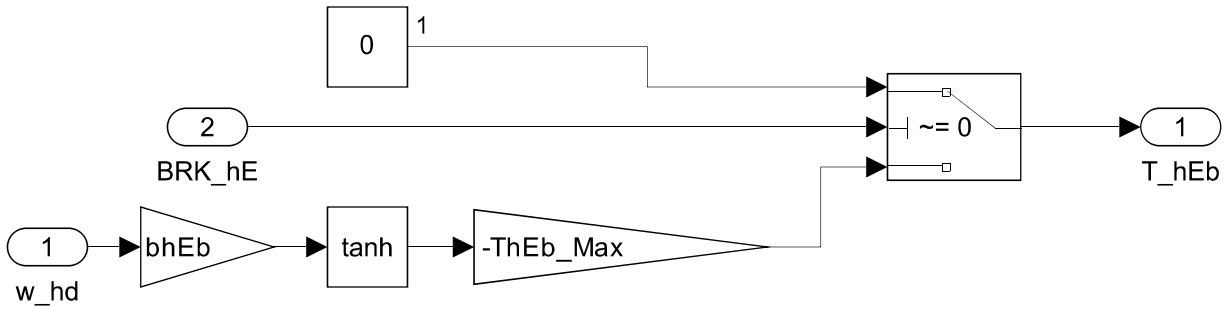

Ttm_Max
1
-Ttm_Max
1
max
min
+
_
1/tau_tm
1
s
1
T_tm
1
T_tm*

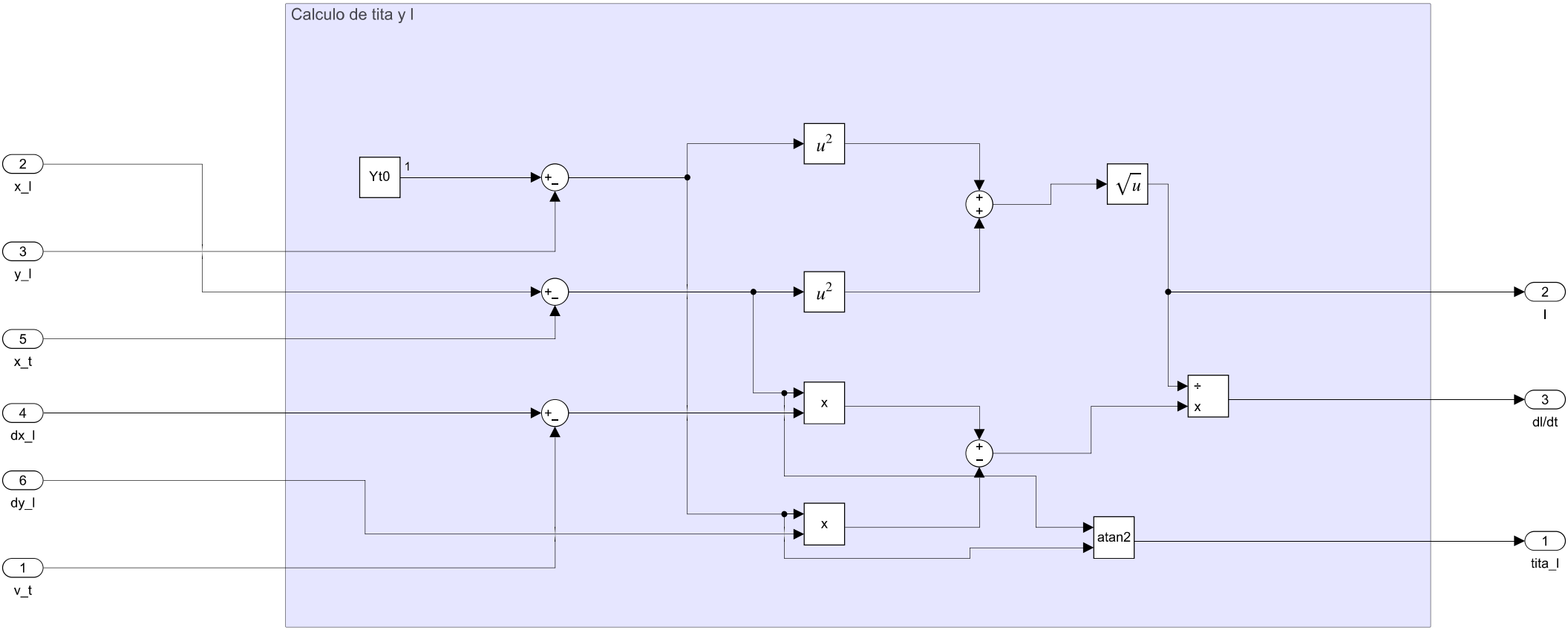

1
1
2*vh_nom*ih/rhd
1
x
÷
u^2
1
w_hm
min
Thm_Max
-1
max
2
T_hm*
min
+_
1/tau_hm
$\frac{1}{s}$
1
T_hm

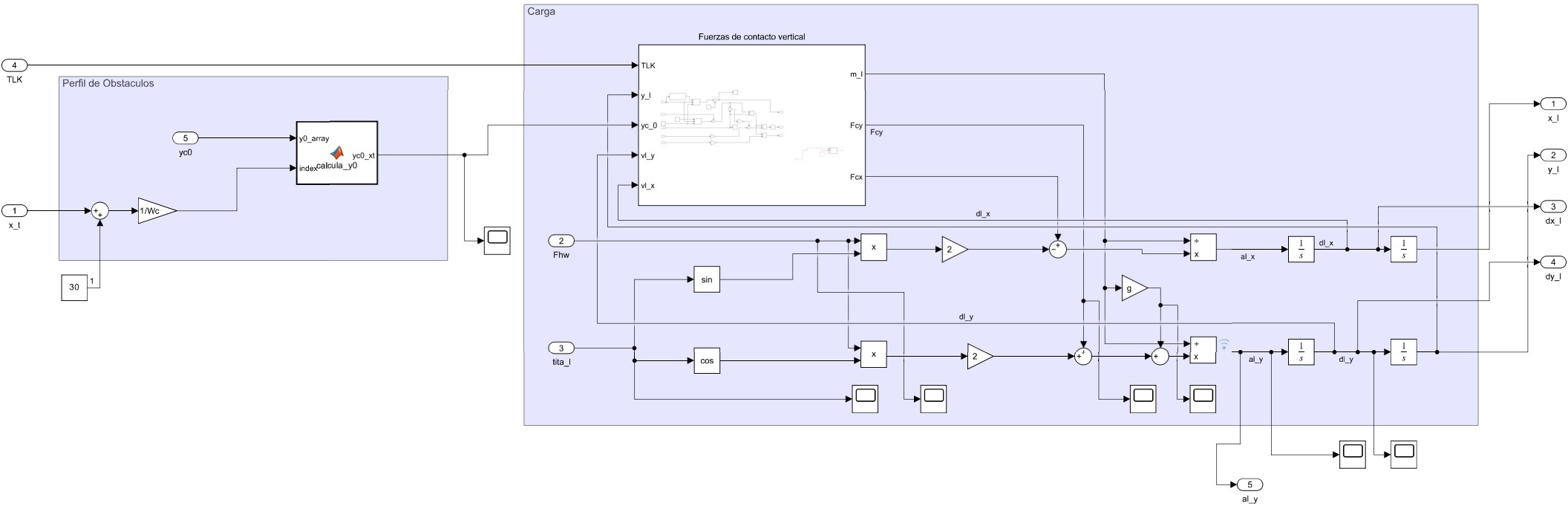

2
BRK_h
1
T_hm
3
BRK_hE
4
F_hw
1/ih
w_hm
BRK_h
T_hb
brake izaje
w_hd
BRK_hE
T_hEb
brake izaje emergencia
1/ih
rhd/ih
-bh_eq
1/Jh_eq
1/s
-rhd/(2*ih)
1/s
1/s
2
w_hm
1
tita_hm
3
l_h
4
dl_h

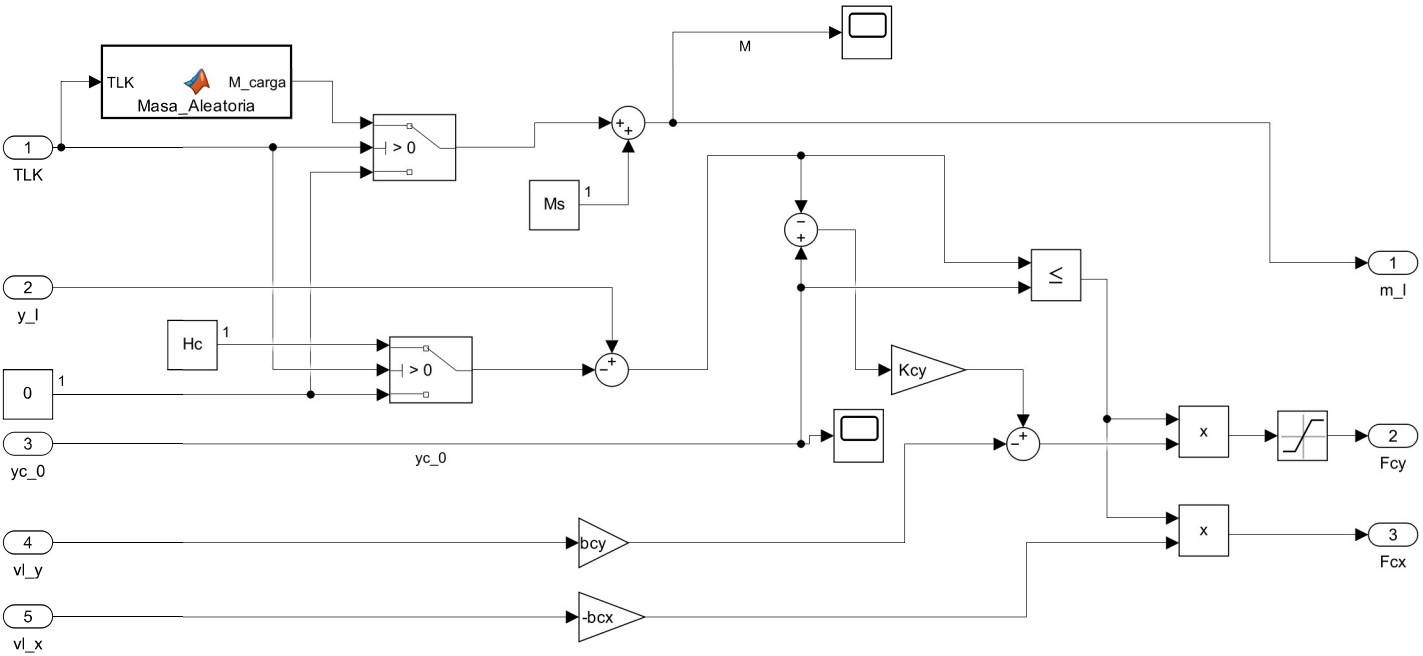
-rhd/(2*Jhd_hEb+2*Jhm_hb*ih^2)


0
1
2
BRK_h
~= 0
1
T_hb
1
w_hm
bhb
tanh
-Thb_Max


0
1
2
BRK_hE
~= 0
1
T_hEb
1
w_hd
bhEb
tanh
-ThEb_Max


Calculo de tita y l
2
x_l
3
y_l
5
x_t
4
dx_l
6
dy_l
1
v_t
Yt0
1
u^2
u^2
x
x
atan2
sqrt(u)
÷
x
2
l
3
dl/dt
1
tita_l


Carga
Fuerzas de contacto vertical
TLK
y_l
yc_0
vl_y
vl_x
m_l
Fcy
Fcx
4
TLK
Perfil de Obstaculos
5
yc0
y0_array
index
calcula_y0
yc0_xt
1
x_l
1/Wc
30
1
2
Fhw
3
tita_l
sin
cos
x
2
g
dl_x
dl_y
al_x
al_y
1
s
1
x_l
2
y_l
3
dx_l
4
dy_l
5
al_y


TLK
M_carga
Masa_Aleatoria
1
TLK
> 0
M
Ms
1
2
y_l
Hc
1
> 0
0
1
3
yc_0
yc_0
Kcy
≤
1
m_l
x
2
Fcy
x
3
Fcx
4
vl_y
bcy
5
vl_x
-bcx


> 0

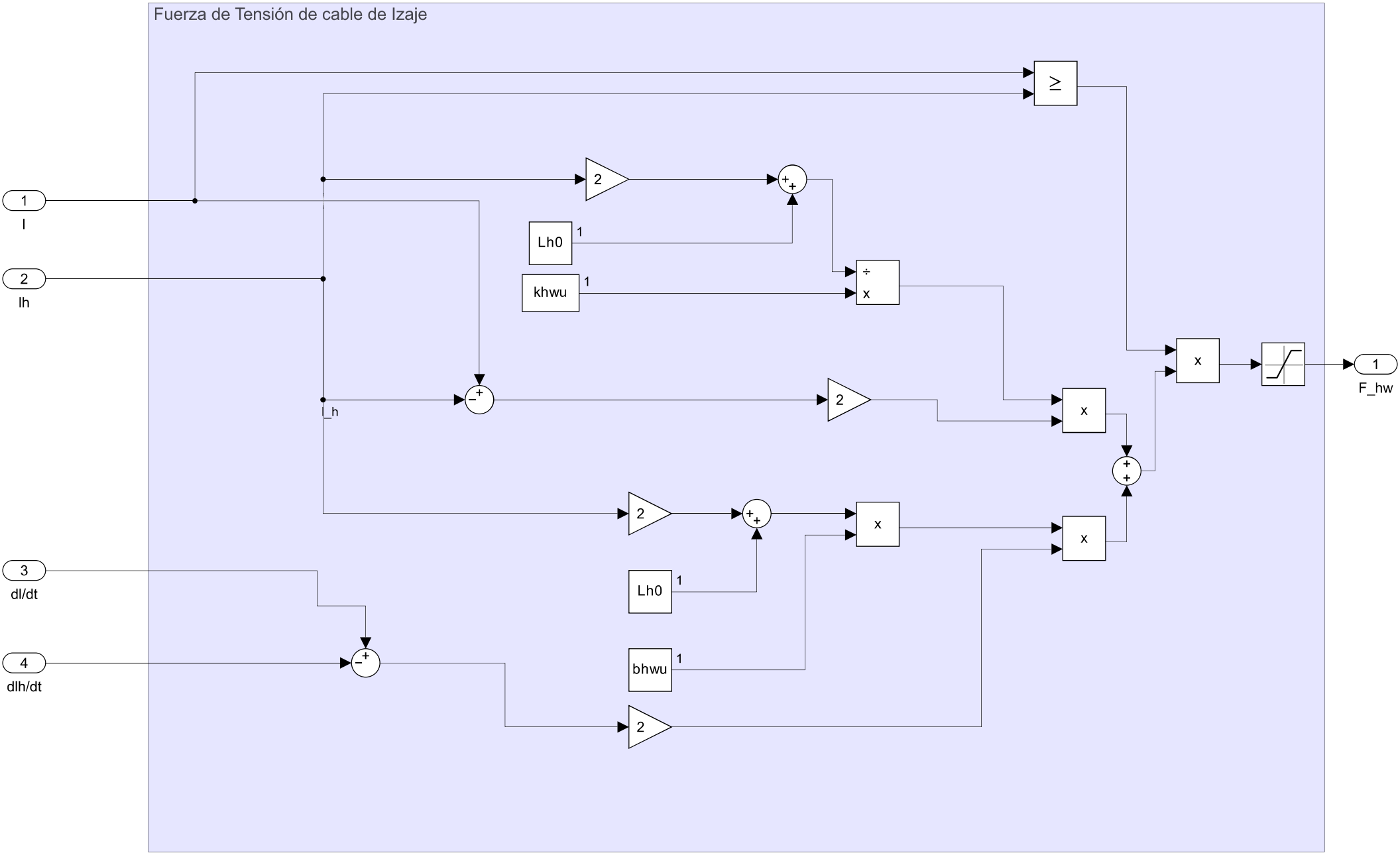
```
function M_carga = Masa_Aleatoria(TLK, Mc_min, Mc_max)
    persistent masa_actual;
    persistent ultimo_estado_trigger;

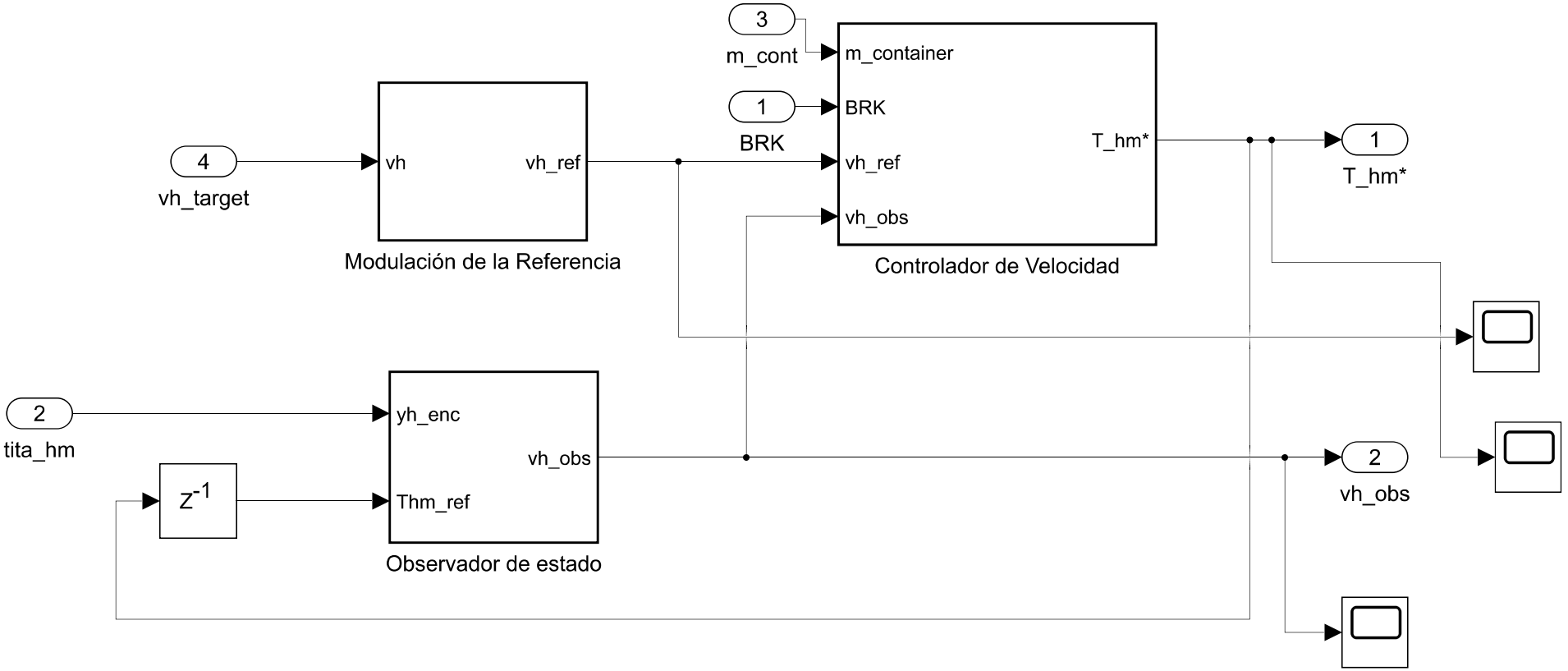
    % Inicialización
    if isempty(masa_actual)
        masa_actual = Mc_min; % Empezamos con el valor mínimo por defecto
        ultimo_estado_trigger = 0;
    end

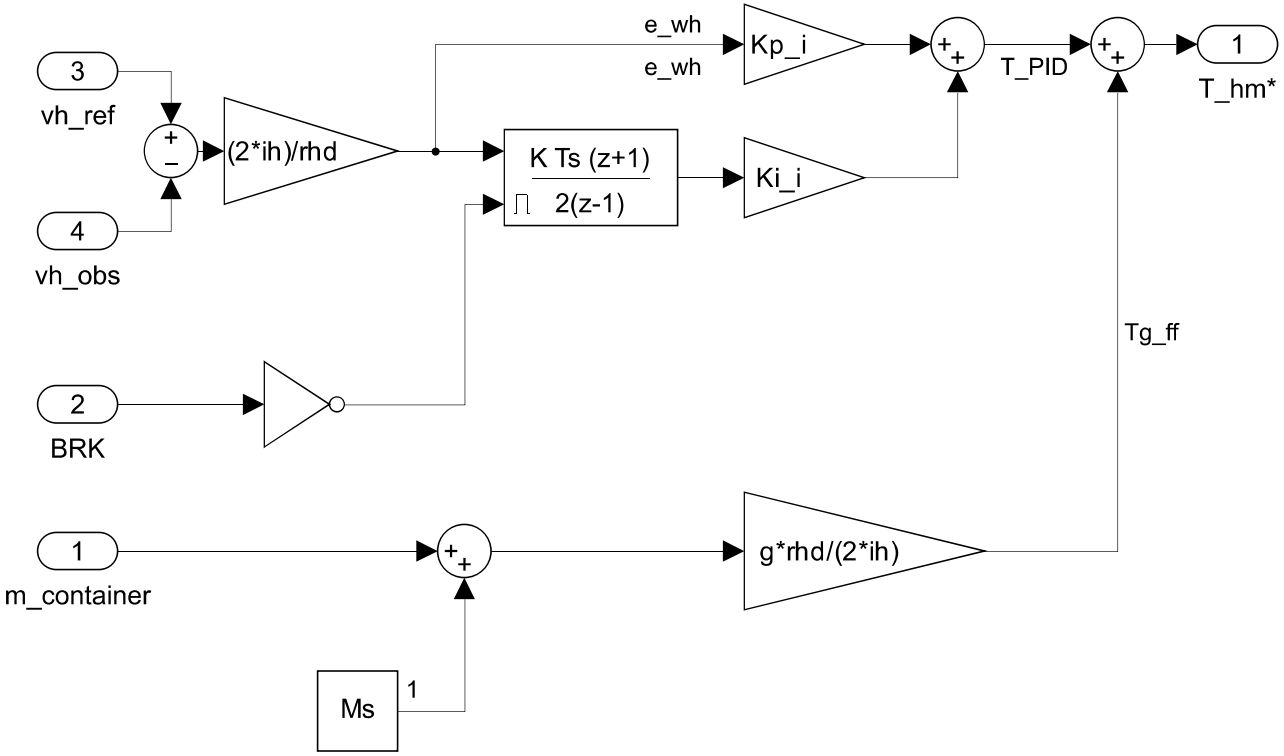
    % DETECCIÓN DE FLANCO DE SUBIDA (Rising Edge)
    if (TLK == 1 && ultimo_estado_trigger == 0)
        % Generar carga aleatoria entre 3tn y 55tn (para tener sobrecarga)
        masa_actual = Mc_min + (Mc_max + 5000 - Mc_min) * rand();
        %masa_actual = Mc_max +1100;
        masa_actual = round(masa_actual);
    end

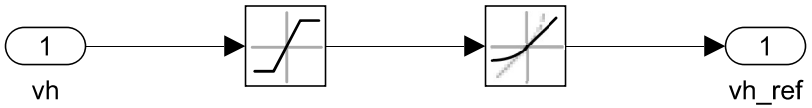
    % Actualizar memoria del trigger
    ultimo_estado_trigger = TLK;
    % Salida: La masa se mantiene constante hasta el próximo trigger
    M_carga = masa_actual;
end
```

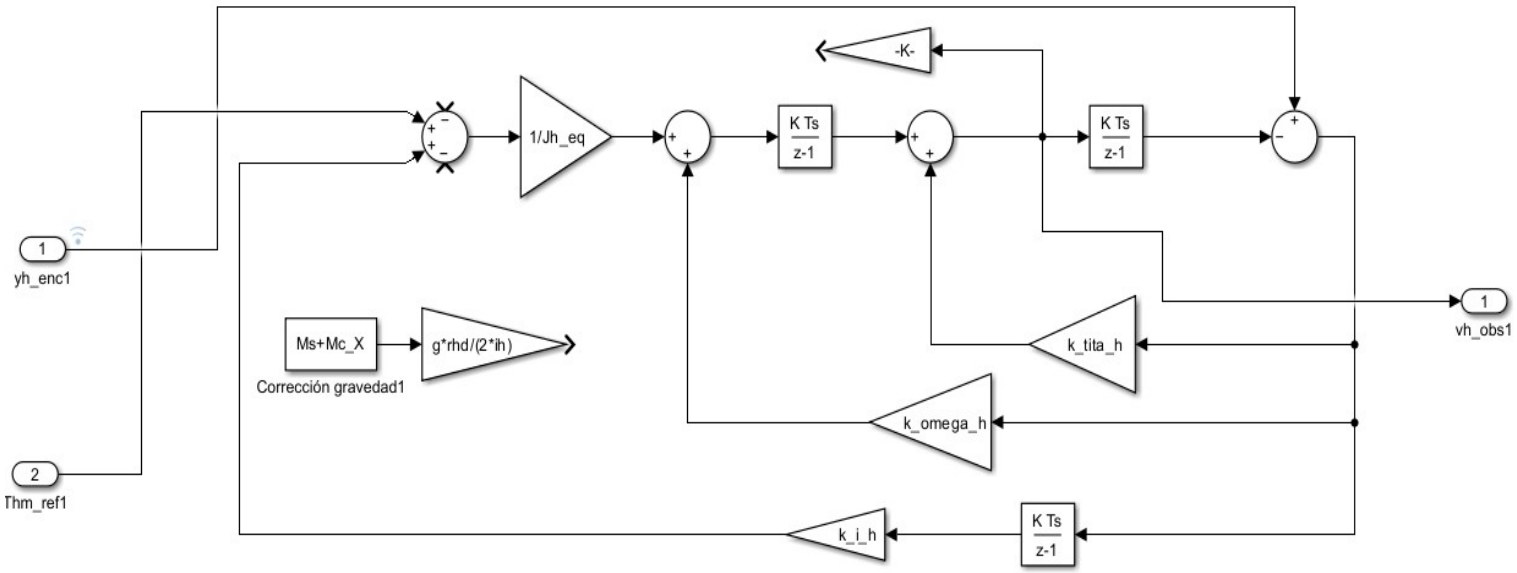
```
function yc0_xt= calcula_y0(y0_array,index)
index = floor(index)+1;
index = min(max(index,1),33);
yc0_xt = y0_array(index);
```

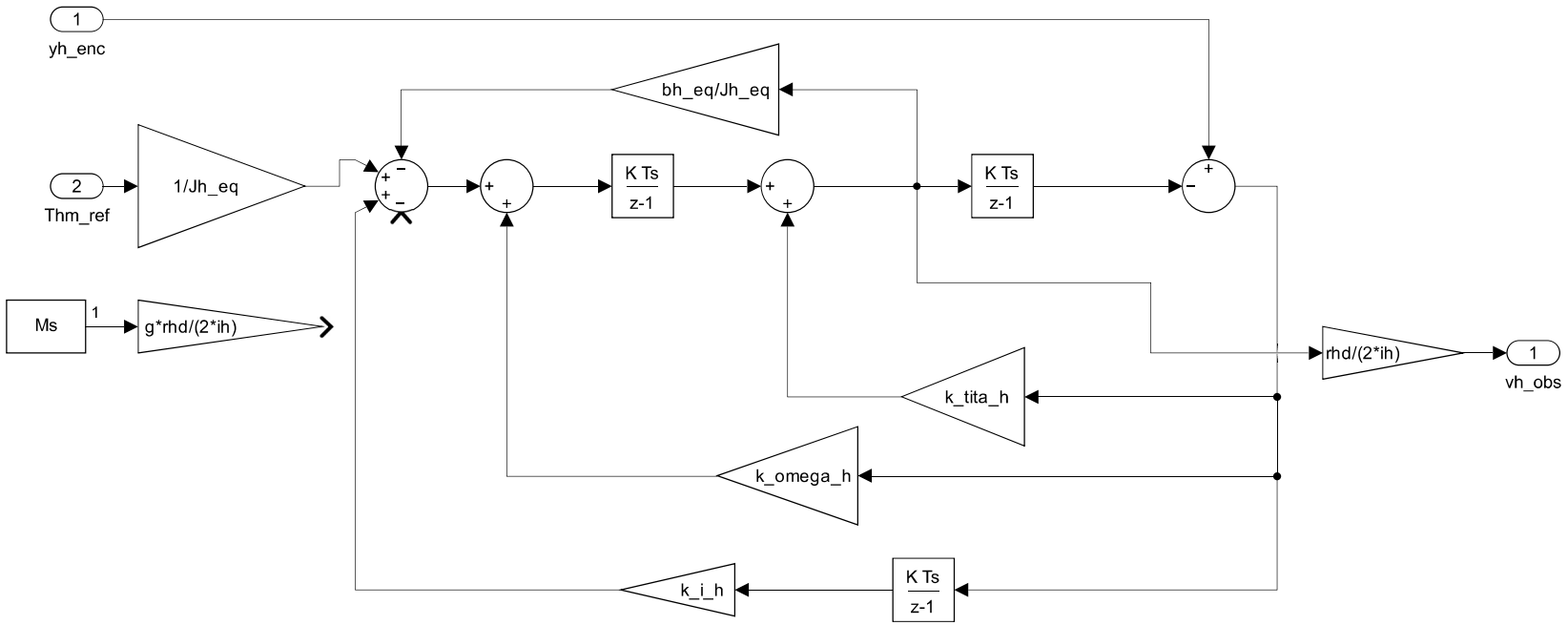

Fuerza de Tensión de cable de Izaje
1
l
2
lh
3
dl/dt
4
dlh/dt
l_h
2
Lh0
1
khwu
1
÷
x
2
x
2
Lh0
1
bhwu
1
x
x
≥
x
x
1
F_hw

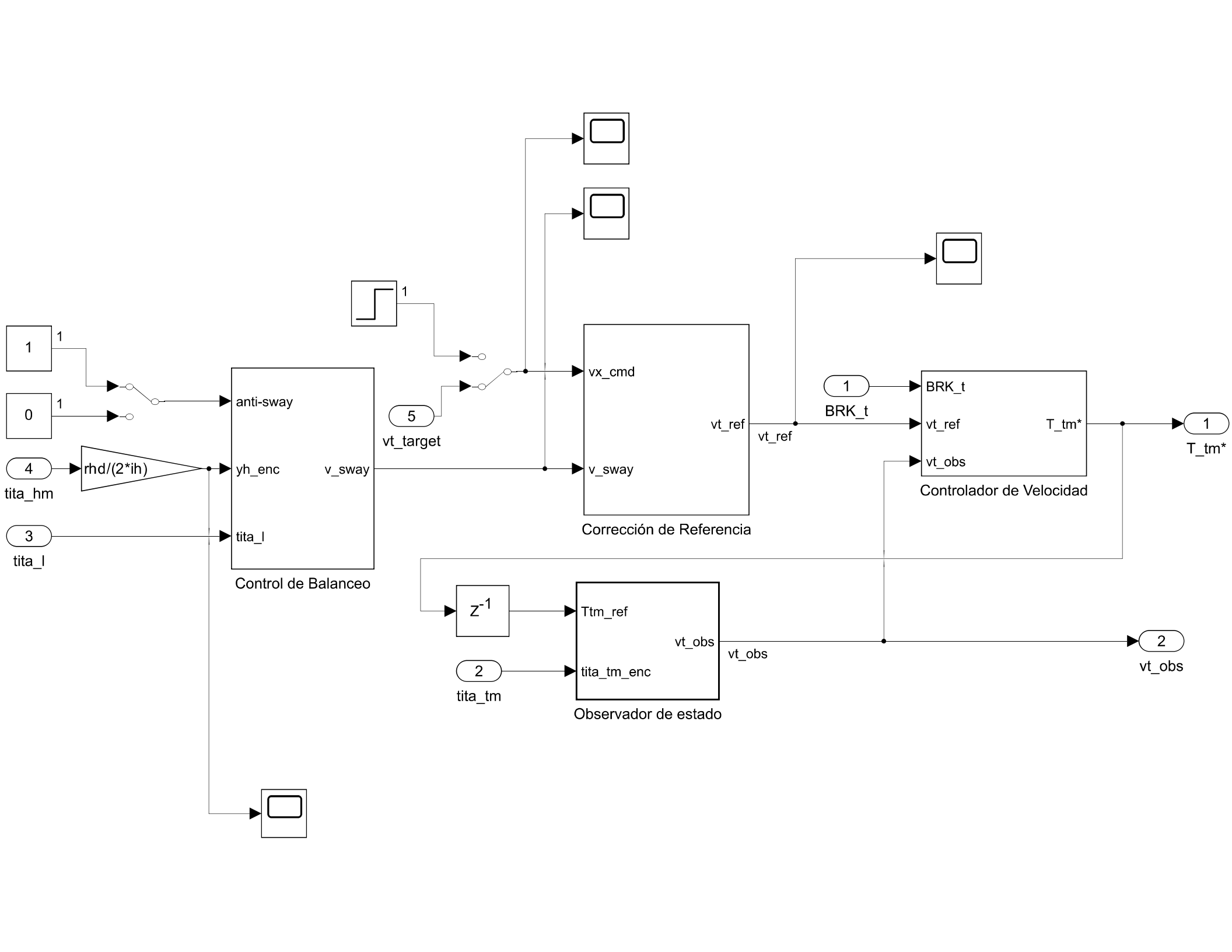

3
m_cont
m_container
1
BRK
BRK
T_hm*
4
vh_target
vh
vh_ref
vh_ref
vh_obs
1
T_hm*
Modulación de la Referencia
Controlador de Velocidad
2
tita_hm
yh_enc
vh_obs
2
vh_obs
Z-1
Thm_ref
Observador de estado

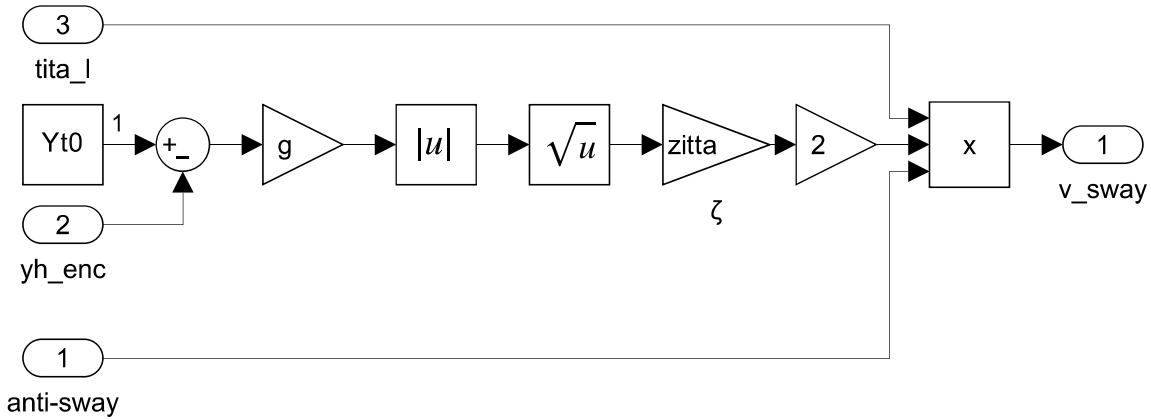

3
vh_ref
4
vh_obs
+
−
(2*ih)/rhd
e_wh
e_wh
Kp_i
K Ts (z+1)
2(z-1)
Ki_i
+
+
T_PID
+
+
1
T_hm*
Tg_ff
2
BRK
1
m_container
+
+
g*rhd/(2*ih)
Ms
1

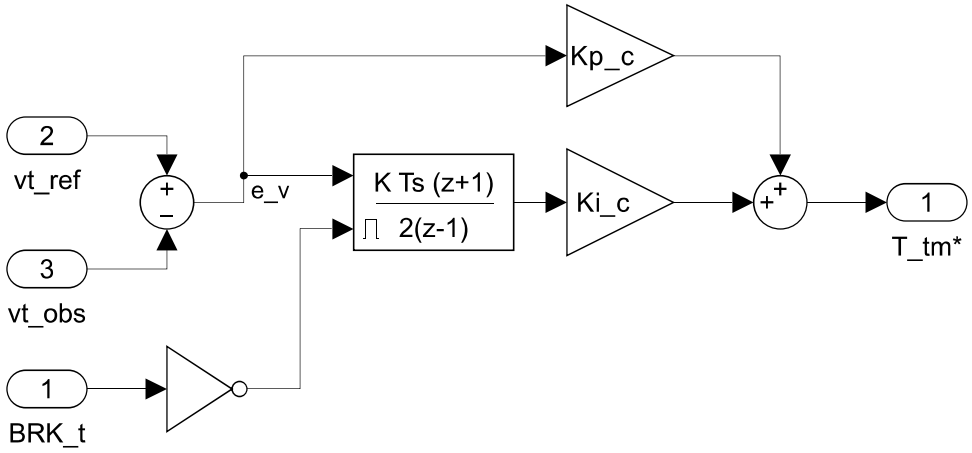

1
vh
1
vh_ref

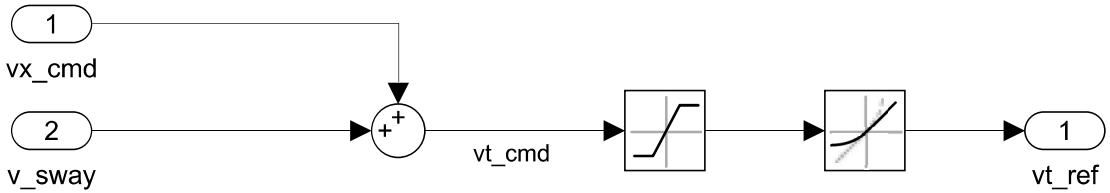

-K-
1/Jh_eq
K Ts
z-1
K Ts
z-1
1
yh_enc1
Ms+Mc_X
g*rhd/(2*ih)
Corrección gravedad1
k_tita_h
1
vh_obs1
k_omega_h
2
Thm_ref1
k_i_h
K Ts
z-1

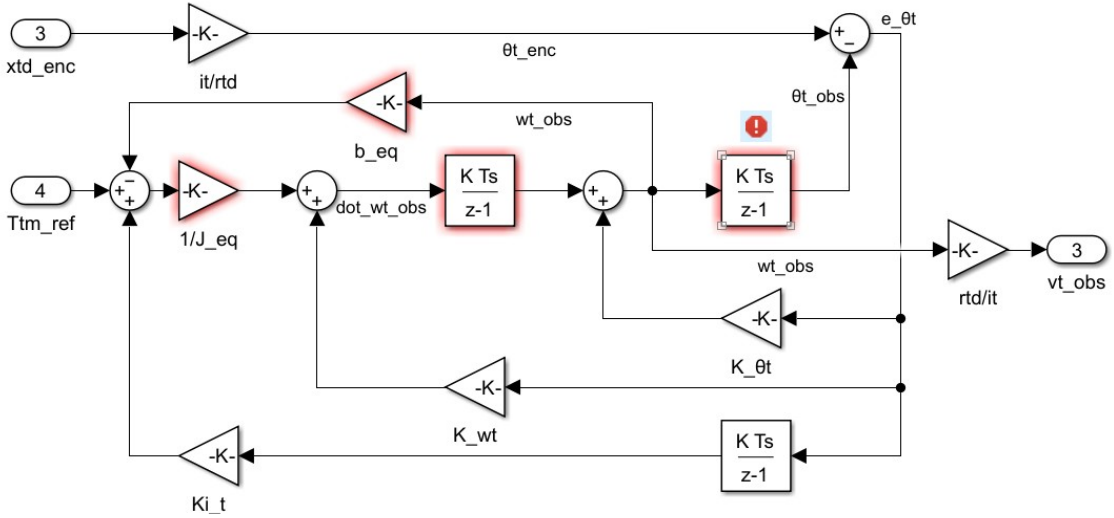

1
yh_enc
bh_eq/Jh_eq
2
Thm_ref
1/Jh_eq
K Ts
z-1
K Ts
z-1
Ms
1
g*rhd/(2*ih)
rhd/(2*ih)
1
vh_obs
k_tita_h
k_omega_h
k_i_h
K Ts
z-1

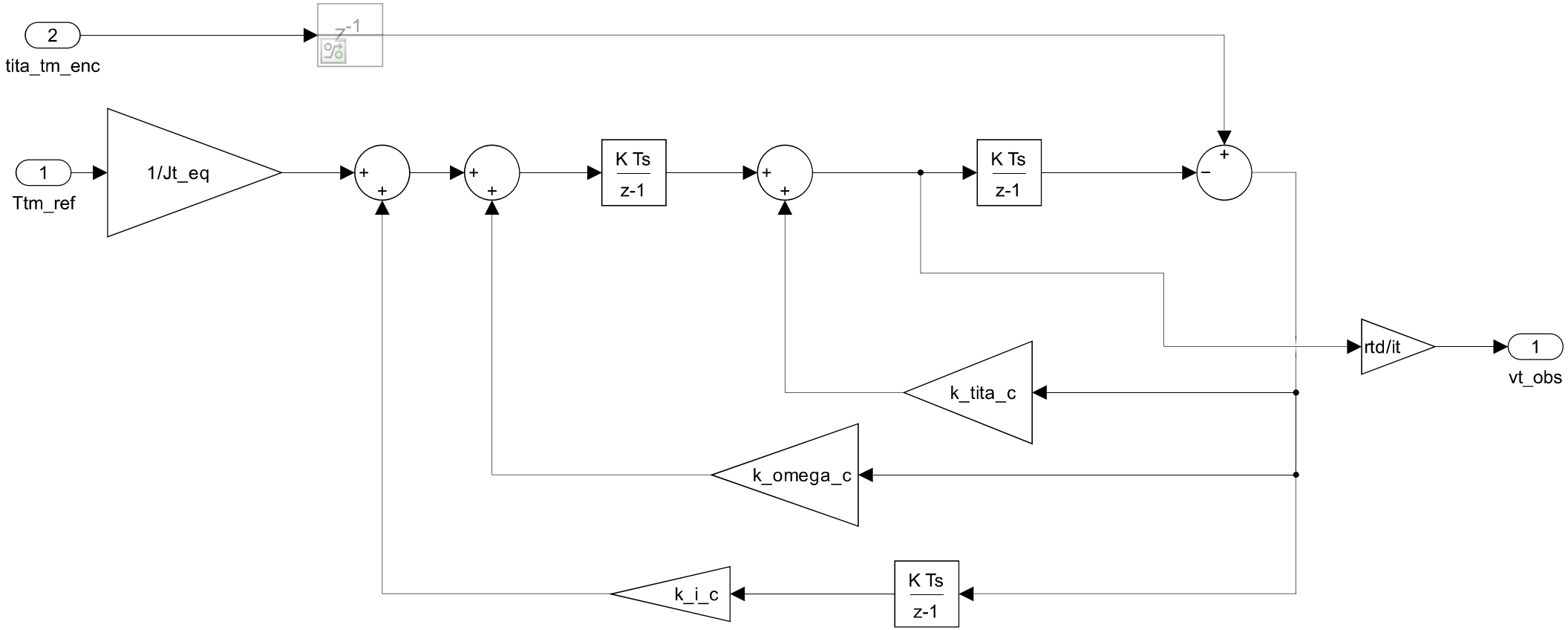

1
1
0
1
4
rhd/(2*ih)
tita_hm
3
tita_l
anti-sway
yh_enc
v_sway
tita_l
Control de Balanceo
1
5
vt_target
vx_cmd
v_sway
vt_ref
Corrección de Referencia
vt_ref
1
BRK_t
BRK_t
vt_ref
vt_obs
T_tm*
Controlador de Velocidad
1
T_tm*
Z-1
Ttm_ref
vt_obs
2
tita_tm_enc
2
tita_tm
Observador de estado
vt_obs
2
vt_obs


3
tita_l
Yt0
1
+_
g
|u|
√u
zitta
ζ
2
x
1
v_sway
2
yh_enc
1
anti-sway


2
vt_ref
3
vt_obs
1
BRK_t
e_v
K Ts (z+1)
2(z-1)
Kp_c
Ki_c
1
T_tm*


1
vx_cmd
2
v_sway
+
+
vt_cmd
1
vt_ref


3
xtd_enc
-K-
it/rtd
θt_enc
e_θt
θt_obs
-K-
b_eq
wt_obs
4
Ttm_ref
-K-
1/J_eq
dot_wt_obs
K Ts
z-1
K Ts
z-1
wt_obs
-K-
rtd/it
3
vt_obs
-K-
K_θt
-K-
K_wt
K Ts
z-1
-K-
Ki_t


2
tita_tm_enc
z-1
1
Ttm_ref
1/Jt_eq
K Ts
z-1
K Ts
z-1
rtd/it
1
vt_obs
k_tita_c
k_omega_c
k_i_c
K Ts
z-1